SYSTEM DESIGN

Design Google Drive / Dropbox

Cloud Storage, Block-Level Sync & Conflict Resolution

~ 35 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Statement & Clarifying Questions
02 · Functional & Non-Functional Requirements
03 · Capacity Estimation
04 · High-Level Architecture
05 · Core Design Decisions
06 · Data Models & Schema
07 · File Upload Pipeline (Write Path)
08 · Storage Architecture & Deduplication
09 · Sync Protocol & Delta Detection
10 · File Download & Reconstruction
11 · Conflict Resolution
12 · Failure Modes & Recovery
13 · Trade-offs & Alternatives
14 · Interview Playbook

01 Problem Statement & Clarifying Questions

Define the scope of the file synchronization and cloud storage system we're designing.

Question	Why It Matters	Typical Answer
Scope: Sync only or full storage?	Determines bandwidth model and complexity; sync implies local copies	Both: cloud storage + device sync
Concurrent users per account?	Affects conflict resolution complexity; shared folders?	Multiple devices per user; shared folders with collaborators
File size limits?	Drives chunking strategy, upload/download protocol	Google: 5TB per file; Dropbox: 2GB (older 350GB for 32-bit)
Real-time sync SLA?	Defines notification latency, polling frequency, cost	Within 30 seconds to 5 minutes acceptable
Offline support?	Local caching strategy, conflict handling	Yes: sync when reconnected; detect conflicts
Sharing & permissions?	Metadata schema complexity, access control	Yes: file/folder sharing, read/write/admin roles
Version history?	Storage and query complexity	Yes: keep last 30 versions (or configurable)
Geographic distribution?	CDN, replication, latency	Global: US, EU, APAC regions; local caching

We'll focus on designing Google Drive/Dropbox: personal cloud storage with multi-device sync, block-level deduplication, and conflict resolution.

02 Functional & Non-Functional Requirements

Functional Requirements

- Upload files: single or chunked multipart (>5MB); resume on failure
- Download files: fetch from cloud or local cache; reconstruct from blocks if needed
- Sync daemon: periodically poll server for changes; detect local file changes
- List files: metadata API with pagination; support folder hierarchy
- Delete files: soft delete with retention period; hard delete after expiry

© Study Guide — For personal use only

- Share files/folders: granular permissions (read, write, admin)
- Version history: keep previous versions; restore to any version
- Conflict resolution: detect concurrent edits; LWW or versioning strategy
- Offline support: continue editing offline; sync when reconnected

Non-Functional Requirements

- **Availability:** 99.99% uptime SLA; graceful degradation if sync service fails
- **Latency:** Sync detection <5 minutes; upload <500ms (metadata); download <1s (metadata)
- **Scale:** 1B+ users; 10PB+ storage; 100K+ concurrent active syncs; 1M+ RPS
- **Bandwidth:** Minimize data transfer via block-level deduplication and delta sync
- **Storage:** Cost-optimized (S3 standard/glacier tiers); quota enforcement per user
- **Security:** Encryption at-rest (AES-256); TLS in-transit; OAuth 2.0 authentication; audit logs
- **Consistency:** Eventual consistency; strong consistency for metadata within same datacenter

03 Capacity Estimation

User & Storage

- **1B+ users worldwide:** ~30% active monthly; ~500M monthly active users
- **Avg storage per user:** ~10GB (Google Drive); ranging 100MB–2TB
- **Total storage:** 500M users × 10GB = 5EB (exabytes) = 5,000 petabytes
- **Deduplication factor:** ~40% (duplicate files, backups, shared folders reduce net storage)
- **Net storage after dedup:** 5EB × 0.6 = 3EB physical storage on S3

Sync Activity

- **Peak concurrent active syncs:** 100K–500K (users actively editing/uploading)
- **Sync frequency:** 5 minute average interval; desktop daemon checks every 30 seconds if idle
- **Files changed per sync cycle:** avg 2–5 files per user per day; 100M file operations/day
- **Bandwidth (sync):** 500M users × 100MB/month (avg change) ÷ 30 days = ~1.7 Gbps baseline
- **Peak bandwidth (uploads):** During business hours, 100K concurrent users × 1Mbps = 100 Tbps

RPS Estimates

- **Upload RPS:** 100K concurrent × 1 req/minute = ~1.7K RPS (baseline); ~10K RPS at peak
- **Download RPS:** ~5K RPS (viewing, sharing downloads)
- **Metadata RPS:** List files, get versions, check permissions: ~50K RPS
- **Sync delta queries:** 100K active syncs × 1 query/5min = ~333 RPS; spike to ~5K RPS
- **Total API RPS:** ~60K RPS baseline; ~100K RPS at peak

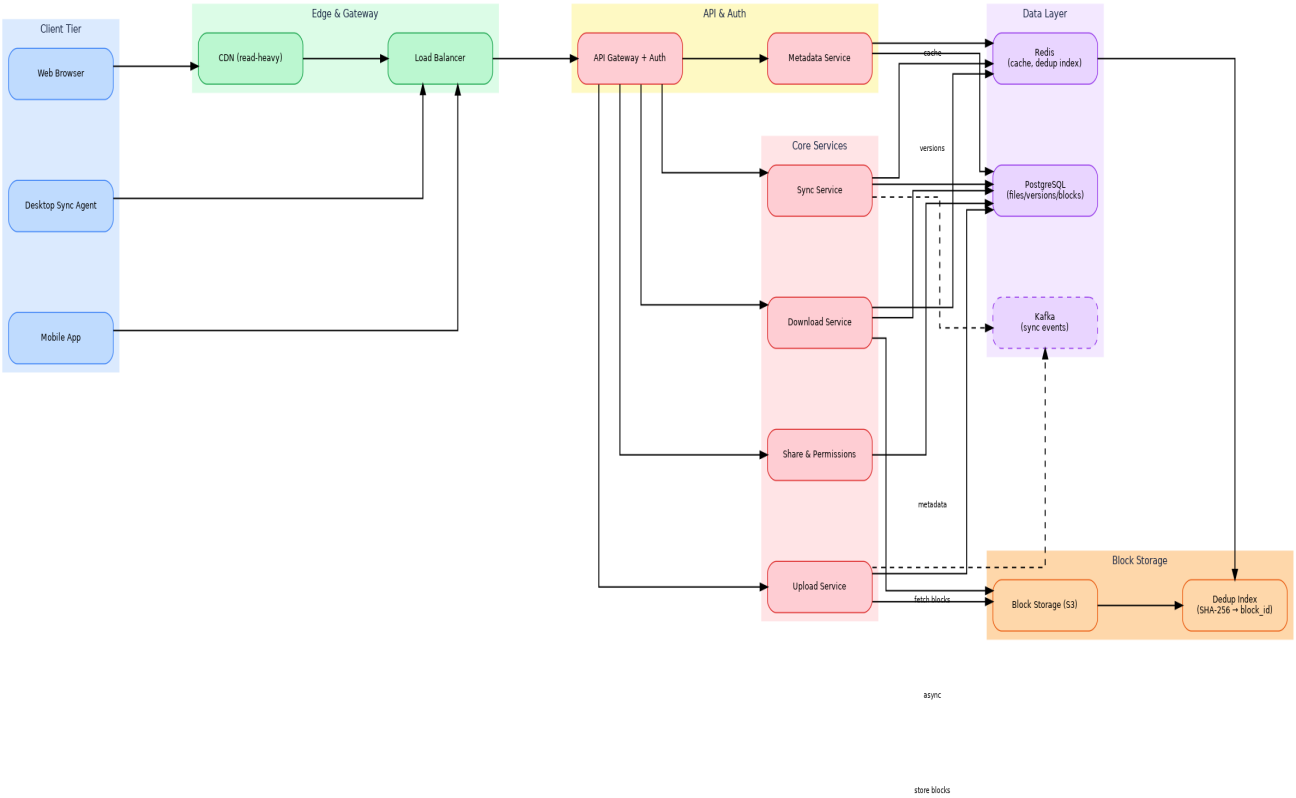
© Study Guide — For personal use only

04 High-Level Architecture

A distributed system serving files with block-level sync, deduplication, and multi-device consistency.

© Study Guide — For personal use only

High-Level System Architecture



 © Study Guide — For personal use only

Client Tier

- **Web client:** Browser-based; OAuth login; upload/download via REST API or resumable upload
- **Desktop sync daemon:** Monitors local folders; detects file changes (inotify on Linux, FSEvents on macOS, USN Journal on Windows)
- **Mobile apps:** iOS/Android; background sync; selective sync (download full folder or streaming preview only)

API Gateway & Load Balancing

- **Load balancer:** Layer 4 (TCP/UDP) + Layer 7 (HTTP); routes to nearest regional gateway
- **API Gateway:** Authentication (OAuth 2.0, MFA); rate limiting per user/IP; request validation; timeout handling
- **Resilience:** Circuit breaker; fallback to replica region if primary unavailable

Microservices

- **Upload Service:** Receives file chunks; validates hash; orchestrates multipart S3 upload; publishes events
- **Download Service:** Reconstructs files from S3 blocks; handles range requests; caching logic
- **Metadata Service:** File metadata (name, size, mtime, owner, permissions); folder hierarchy; versioning
- **Sync Service:** Delta detection; compares client's last-known state with server; returns changed file list and block indices
- **Auth Service:** OAuth token validation; permission checks; audit logging
- **Notification Service:** WebSocket push or long-polling; notifies clients of changes in real-time

Storage & Caching

- **S3 Blob Store:** Stores file blocks (4MB chunks); each block identified by content hash (SHA-256 deduplication)
- **PostgreSQL Metadata DB:** File metadata, folder structure, versions, permissions; strong consistency for correctness
- **Redis Cache:** File index (lookup file_id by path), user quota, popular blocks, session data; 99% hit rate target

Background Processing

- **Block Processor:** Validates hashes, detects duplicates, compresses blocks, deduplicates across users
- **Thumbnail Generator:** Generates previews for images, PDFs, docs; stores in separate S3 bucket
- **Delta Sync Service:** Asynchronous block-level synchronization for cross-platform devices

Message Queue

© Study Guide — For personal use only

- **Kafka:** Decouples upload service from async processing; topics: file.uploaded, file.deleted, file.shared; enables event sourcing and audit trail

05 Core Design Decisions

Block-Level Sync vs. Full File Sync

- **Block-level approach (chosen):** Files split into 4MB blocks; only changed blocks re-uploaded; massive bandwidth savings (10–50x)
- **Example:** 1GB document with 1 paragraph change → upload only changed 4MB block, not entire 1GB
- **Trade-off:** Complexity in tracking block versions, reconstructing files, deduplication logic

Deduplication Strategy

- **Content-addressed storage:** Block hash (SHA-256) is the key; identical blocks across users/files stored once
- **Dedup ratio:** Typical 40–50% reduction (backup files, duplicate documents, similar media)
- **Trade-off:** Hash collisions rare but need validation; storage overhead for hash index

Delta Sync Protocol

- **Client state tracking:** Daemon maintains cursor (sequence number) of last-synced state on server
- **Server comparison:** On sync request, server returns only files changed since cursor
- **Block-level delta:** For large files, only return changed block indices (not full file contents)
- **Advantage:** <50ms metadata overhead; minimal bandwidth for unchanged files

Conflict Resolution Strategy

- **Last-Write-Wins (LWW):** Default; compare modification time (mtime); newer version wins
- **Versioning fallback:** If mtime equals, content differs (concurrent edits) → rename local as 'conflict'; keep server version
- **User opt-in versioning:** Paid plans: keep all versions, restore any version; free plans: limited version history
- **Conflict detection:** Server tracks version vector per file; client reports local mtime/hash

06 Data Models & Schema

PostgreSQL schema for metadata; S3 for blocks.

```
CREATE TABLE files (
```


© Study Guide — For personal use only

```
file_id UUID PRIMARY KEY,
owner_id BIGINT NOT NULL,
name VARCHAR(255) NOT NULL,
path VARCHAR(2048) NOT NULL,
mime_type VARCHAR(100),
size_bytes BIGINT,
created_at TIMESTAMP,
modified_at TIMESTAMP,
deleted_at TIMESTAMP,
current_version_id UUID,
FOREIGN KEY (owner_id) REFERENCES users(user_id),
UNIQUE (owner_id, path)
);

CREATE TABLE blocks (
block_hash CHAR(64) PRIMARY KEY, -- SHA-256 hex
size_bytes INT,
s3_key VARCHAR(255) NOT NULL,
created_at TIMESTAMP,
last_access_at TIMESTAMP,
dedup_count INT DEFAULT 1 -- how many files reference this block
);

CREATE TABLE file_blocks (
file_version_id UUID NOT NULL,
block_sequence INT NOT NULL,
block_hash CHAR(64) NOT NULL,
FOREIGN KEY (block_hash) REFERENCES blocks(block_hash),
PRIMARY KEY (file_version_id, block_sequence)
);

CREATE TABLE versions (
version_id UUID PRIMARY KEY,
file_id UUID NOT NULL,
created_at TIMESTAMP,
modified_at TIMESTAMP,
FOREIGN KEY (file_id) REFERENCES files(file_id)
);

CREATE TABLE user_quota (
user_id BIGINT PRIMARY KEY,
quota_bytes BIGINT DEFAULT 15737418240, -- 15GB
used_bytes BIGINT DEFAULT 0,
FOREIGN KEY (user_id) REFERENCES users(user_id)
```

 © Study Guide — For personal use only

```
);
```

PostgreSQL Schema (simplified)

Key Design Decisions

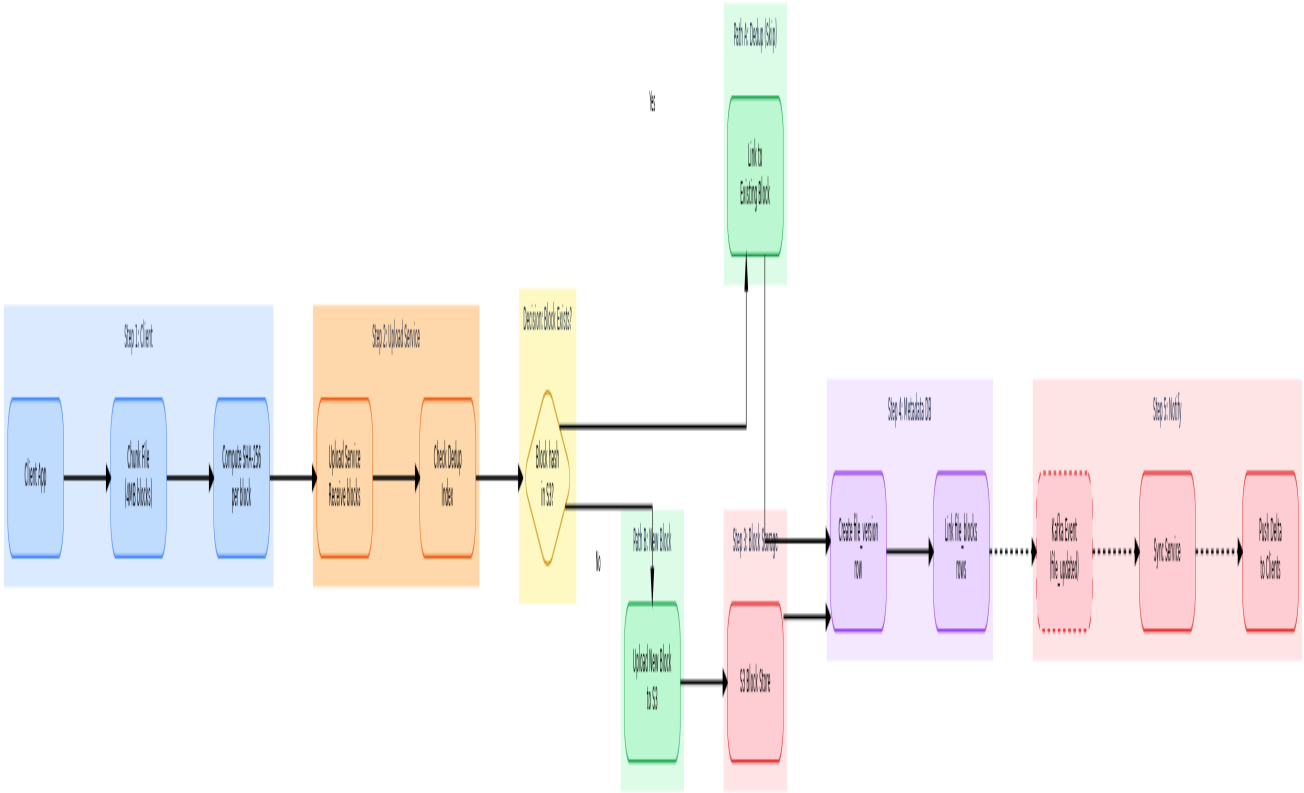
- **Content addressing (block_hash):** Enables deduplication; block stored once per unique hash
- **Version history:** Separate versions table; each version has reference to file_blocks list
- **Soft delete:** deleted_at timestamp; cleanup job removes after 30 days (configurable)
- **Indexing:** (owner_id, path) unique index for fast file lookups; block_hash index for dedup checks

07 File Upload Pipeline (Write Path)

Chunked multipart upload with block deduplication and async processing.

© Study Guide — For personal use only

File Upload & Block Deduplication Pipeline



 © Study Guide — For personal use only

Small File Upload (<5MB)

- **Protocol:** PUT /files/{file_id}; single HTTP request
- **Flow:** Client → API Gateway → Upload Service → S3 (direct put) → return 200 OK
- **Latency:** <100ms metadata; upload time depends on file size + network

Large File Upload (>5MB)

- **Protocol:** POST /files/{file_id}/upload (initiates); PUT /files/{file_id}/chunk/{seq} (upload chunks); POST /files/{file_id}/finalize (commits)
- **Chunking:** Split into 4MB blocks client-side; compute SHA-256 hash per block
- **Parallel upload:** Client uploads up to 4 blocks in parallel for bandwidth efficiency
- **Multipart resume:** If connection drops, retry failed blocks; server tracks ETag per part

Deduplication & Block Processing

- **On upload finalize:** Upload Service checks block hashes against PostgreSQL blocks table
- **If block exists:** Increment dedup_count; link file_version to existing block (no S3 re-upload)
- **If block new:** Store in S3 with key based on hash; insert into blocks table
- **Background job (Kafka → Block Processor):** Validates hash integrity; compresses block if beneficial (text > images); cleanup orphaned blocks

Metadata & Event Publishing

- **After finalize:** Create file version record; update files table (size, mtime, current_version_id)
- **Publish event:** 'file.uploaded' to Kafka topic; includes file_id, owner_id, changed blocks
- **Notification Service:** Subscribes to Kafka; pushes change notification to other devices via WebSocket
- **Sync daemon on other devices:** Receives push; wakes up; queries Sync Service for delta

08 Storage Architecture & Deduplication

S3 Blob Storage

- **Bucket structure:** s3://user-files-prod/{block_hash[:2]}/{block_hash[2:]} (hash partitioning for scalability)
- **Block size:** 4MB typical; smaller for thumbnails (100KB), larger for video (10MB)
- **Storage classes:** S3 Standard for hot data (<7 days old); S3 Intelligent-Tiering or Glacier for cold data
- **Encryption:** Server-side encryption (SSE-S3); optional client-side encryption (zero-knowledge backup)
- **Versioning:** S3 versioning disabled (we manage versions in PostgreSQL); lifecycle policies auto-delete old versions after 30 days

 © Study Guide — For personal use only

Deduplication Strategy

- **Content-addressed:** Block key = SHA-256(block_data); identical data → same key → stored once
- **Dedup benefits:** 40–50% storage savings; cost reduction proportional to dedup ratio
- **Dedup cross-user:** Same file backed up by multiple users → 1 block copy; careful with privacy (no metadata correlation)
- **Dedup detection:** On upload, check if block_hash in PostgreSQL; if yes, reuse S3 key
- **Garbage collection:** Periodic job counts live references; delete unreferenced blocks after 30-day grace period

Quota & Cost Management

- **User quota:** Stored in Redis + PostgreSQL; enforced before upload (reject if quota exceeded)
- **Billing:** Charge for logical bytes (before dedup) or physical bytes? Google/Dropbox charge logical (user-visible); incentivizes keeping unique data
- **Cost per GB:** S3 ~$0.023/GB/month; after 40% dedup, effective cost ~$0.014/GB/month

09 Sync Protocol & Delta Detection

Client-Side Sync Daemon

- **Polling interval:** Every 30 seconds (if app in foreground); 5 minutes (if backgrounded)
- **Local change detection:** File system events (inotify, FSEvents); track local file mtime + size hash
- **State storage:** .sync_state.db (SQLite) on client; stores file metadata at last sync
- **Offline support:** Queue changes locally; sync when reconnected; detect conflicts during reconciliation

Sync Service Delta Detection

- **Cursor-based pagination:** Client maintains cursor (e.g., version_id or timestamp); requests changes since cursor
- **Query:** SELECT files WHERE modified_at > @cursor AND owner_id = @user_id ORDER BY modified_at LIMIT 1000
- **Response:** List of changed files; includes file_id, name, size, mtime, current_version_id, deleted flag
- **Block-level delta:** For large files (>10MB), return only block indices changed, not full file contents
- **Latency:** <50ms query + network round-trip

Notification Service

- **WebSocket connection:** Daemon opens persistent WebSocket to Notification Service; maintains connection for push
- **On-file-change event:** Kafka publishes file.uploaded event; Notification Service pushes to all watching clients in real-time

© Study Guide — For personal use only

- **Fallback (if WebSocket fails):** Client retries HTTP long-polling; Notification Service holds request open for up to 30s waiting for changes
- **Advantage:** Real-time sync; users see changes within <1s on multiple devices

10 File Download & Reconstruction

Small Files (<4MB Single Block)

- **Protocol:** GET /files/{file_id}
- **Flow:** Download Service fetches block from S3; stream to client; latency ~100ms
- **Caching:** Redis caches frequently accessed file blocks; 99% cache hit rate for popular files

Large Files (Multi-Block)

- **Protocol:** GET /files/{file_id}; supports Range header for resumable downloads
- **Reconstruction:** Download Service queries file_blocks for file_version_id; fetches blocks in parallel from S3
- **Streaming:** As blocks arrive, stream to client immediately (don't wait for all blocks)
- **Latency:** <100ms for metadata; block fetch parallelized (4–8 parallel S3 Gets)

Sync Daemon Download

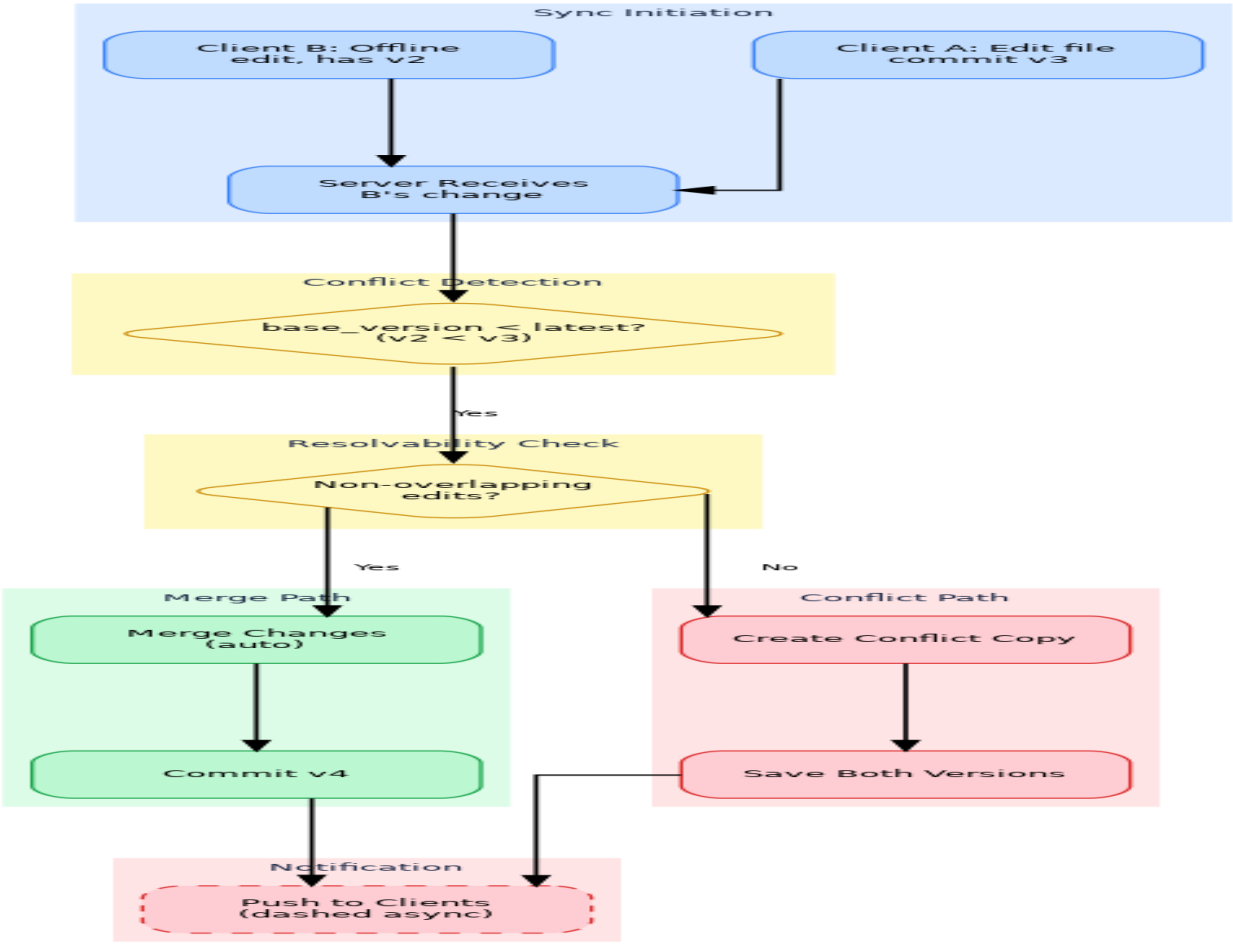
- **Selective sync:** User can choose which folders to sync locally; download only selected blocks
- **Smart sync (cloud-only):** File metadata synced, but file contents remain in cloud; fetch on-demand when accessed
- **Conflict handling:** If local file modified during download, abort and retry after conflict resolution

11 Conflict Resolution

How to handle concurrent edits across devices.

© Study Guide — For personal use only

File Sync & Conflict Resolution Flow



 © Study Guide — For personal use only

Conflict Detection

- **Trigger:** During sync, client reports local file mtime + content hash; server compares against current_version in PostgreSQL
- **Mismatch:** Server mtime > client mtime → server version newer; mtime equal but hash differ → concurrent edit
- **Metadata conflict:** File renamed on server but also on client locally → detect and resolve

Last-Write-Wins (LWW) Resolution

- **Rule:** Compare modification timestamps (UNIX epoch, second precision)
- **Server timestamp > client timestamp:** Accept server version; overwrite local file
- **Client timestamp > server timestamp:** Local version is newer; upload to server; overwrite cloud version
- **Pros:** Simple; deterministic; no user confusion
- **Cons:** Data loss if both versions have value; not suitable for collaborative docs

Versioning-Based Conflict Resolution

- **For premium users:** Keep all versions; on conflict, rename local as '{filename}.conflict.{timestamp}'
- **Example:** 'Document.docx' on server (v5); 'Document.docx' on client (v4, modified); create 'Document.conflict.1712345678.docx' locally; pull server v5
- **User action:** Manually merge conflicted file; delete conflict version; re-upload
- **Pros:** No data loss; users can merge manually
- **Cons:** Manual effort; messy for large teams

Operational Transform (OT) / CRDT (Rare)

- **For collaborative real-time editing:** Use CRDT (Conflict-free Replicated Data Type) or OT; not standard for file sync
- **Example:** Google Docs uses OT for character-by-character merging; not feasible for binary files
- **Dropbox Smart Sync:** For .docx, .xlsx: integrates with apps via API for true collaboration

12 Failure Modes & Recovery

Failure	Detection	Recovery
S3 outage	Upload returns 500; Download → fallback to cache	Exponential backoff; retry queue; fallback region S3
Metadata DB unavailable	Connection timeout; circuit breaker opens	Replica read-only; fallback to cached metadata; queue writes for later

© Study Guide — For personal use only

Failure	Detection	Recovery
Upload interrupted	Client detects TCP reset; 0 bytes received	Resume from last confirmed block using ETag; re-upload from checkpoint
Sync daemon crash	No heartbeat from device; push notification undelivered	Daemon auto-restarts; on next launch, full delta sync from cursor
Block hash collision	Two blocks with same hash but different content (1 in 2^256 per block)	Cryptographic validation on read; store both as separate entries
Quota exceeded	User hits 15GB limit; next upload rejected	Soft enforcement; block further uploads; charge for overage or auto-delete old versions
Concurrent delete & upload	File deleted on server while client uploading	Accept upload → new file version created; client sees no local file on next sync
Shared folder permission change	Revoke access; user still has cached copy locally	Daemon detects access denied on next sync; remove local copy; audit log entry

13 Trade-offs & Alternatives

Decision	Alternative	Why Chosen
4MB block size	1MB (more granular) or 64MB (fewer metadata)	Balance: overhead for small files; efficiency for large files
SHA-256 hash	MD5 (faster) or CRC32 (simpler)	Security + collision resistance; CPU cost negligible with HW acceleration
LWW resolution	Versioning or CRDT	Simplicity; most conflicts are non-critical; users rarely edit simultaneously
PostgreSQL metadata	DynamoDB or Cassandra	ACID guarantees for consistency; acceptable latency; schema flexibility for schema evolution
S3 storage	Custom object store	Cost: S3 11¢/GB vs. custom ~1¢/GB; but custom requires operational overhead
Eventual consistency	Strong consistency	Availability & latency >> perfect consistency; 5 minute sync window acceptable
Kafka async processing	Synchronous	Decouple upload from thumbnail generation; improve perceived latency to user

© Study Guide — For personal use only

Decision	Alternative	Why Chosen
WebSocket + long-polling	Polling only (5 min interval)	Real-time sync drives user engagement; offset by lower polling frequency when idle

14 Interview Playbook

How to Present This Design (45 Minutes)

Opening (5 min): Clarify & Scope

- Ask 2–3 questions: Sync + storage? Multi-device? Sharing? Version history?
- Confirm scale: 1B users, 10PB storage, 100K concurrent syncs
- Set scope: Focus on core sync + upload/download; mention but don't deep-dive sharing/versioning

Problem (3 min): Capacity Math

- **Users & storage:** 500M active × 10GB = 5EB logical; 3EB after 40% dedup
- **Sync load:** 100K concurrent syncs × 5 min interval = 333 RPS baseline
- **Bandwidth:** Block-level delta sync saves 90% vs. full file sync

Architecture (10 min): Tiers & Flow

- **Client tier:** Web, desktop daemon (local FS monitoring), mobile (background sync)
- **API gateway:** Auth, rate limiting, request routing
- **Microservices:** Upload, Download, Metadata, Sync, Notification (draw on board!)
- **Storage:** S3 blocks + PostgreSQL metadata + Redis cache
- **Async:** Kafka topics; Block Processor for dedup, Thumbnail Generator, Delta Sync Service

Deep Dives (20 min): Pick 2 Topics

- **Option 1 (Upload):** Chunked multipart; block deduplication; parallel uploads; finalize with metadata
- **Option 2 (Sync):** Cursor-based delta detection; block-level diffs; WebSocket push notifications
- **Option 3 (Conflict Resolution):** LWW vs. versioning; compare mtimes + hashes; user-facing conflict handling
- **Option 4 (Deduplication):** Content-addressed storage; 40–50% space savings; hash index strategy
- **Option 5 (Failure Recovery):** Interrupted uploads (resume); metadata loss (replicas); network partitions (eventual consistency)

Wrap-Up (5 min): Trade-offs

- Block-level sync vs. full file: complexity worth it for bandwidth savings
- Eventual consistency vs. strong: acceptable for file sync; 5 min window is fine

© Study Guide — For personal use only

- Custom storage vs. S3: S3 easier operationally; cost tradeoff worth it
- End: 'Design driven by scale; block sync is key differentiator'

Key Numbers to Memorize

- 1B users, 500M monthly active; 10PB storage, 3EB after dedup (40% factor)
- 100K concurrent active syncs; 333 RPS (5 min interval); 100K RPS at peak upload
- 4MB block size; SHA-256 content hash; LWW conflict resolution
- 99% cache hit rate (Redis); 99.99% availability SLA
- WebSocket push + long-polling fallback; <1s notification latency

What Interviewers Test

- **Block-level thinking:** Why 4MB blocks? What about dedup? Reconstruction overhead?
- **Sync protocol:** How to detect changes? What about offline devices? Conflicts?
- **Consistency model:** Why eventual? What are edge cases? How to handle concurrent edits?
- **Scale intuition:** 1B users, 10PB storage; how does that affect each component?
- **Trade-offs:** Why not use S3 versioning? Why custom daemon? Why 4MB not 1MB?

© Study Guide — For personal use only

N KEY TAKEAWAYS

1. Block-level sync is the key differentiator: Small file changes trigger only 4MB block uploads, not whole-file transfers. Bandwidth savings of 90%+ for typical workflow.

2. Content-addressed storage enables deduplication: Hash-based block keys allow single storage copy across users and files. 40–50% space savings justified the complexity.

3. Delta-sync protocol: Cursor-based queries + version vectors detect changes in <50ms. Combines efficiency with real-time notifications via WebSocket.

4. Conflict resolution is pragmatic: LWW (Last-Write-Wins) on mtime works for most cases. Versioning feature for premium users who need no-data-loss guarantee.

5. Async processing critical: Kafka decouples upload from thumbnail generation, dedup validation, block compression. Improves user-facing latency.

© Study Guide — For personal use only

Additional Resources & Reading

- **Dropbox Tech Blog:** 'Building Better File Sync' (details on efficient sync protocol)
- **Google Drive Architecture:** Official blog posts on block-level dedup and multi-device sync
- **System Design Interviews:** Practice explaining chunked upload, delta sync, conflict resolution until fluent
- **Real-world complexity not covered:** Shared folders (ACL complexity), mobile offline mode (local DB sync), encryption (key management), compliance (GDPR retention policies)

© Study Guide — For personal use only